

Laboratorio 8 — DuckDB

Informe breve de resultados

CC3084 Data Science · Universidad del Valle de Guatemala · Ciclo 2, 2026

Equipo (usuarios de GitHub): Nesstor07, ecarcamo y BARHUG0

Repositorio: github.com/ecarcamo/duckdb (rama lab8)

1. Entregables del repositorio

Los datos descargados no están en Git (data/raw y data/processed están en .gitignore); el README explica cómo levantar el ambiente, descargar los datos, ejecutar el análisis, reproducir el benchmark y regenerar el tablero.

Requisito	Ubicación en el repositorio
Código fuente modificado y scripts de descarga	scripts/download_data.py (años con --years, verificación con --verify), scripts/taxi_db.py, docker-compose.yml
Consultas SQL	sql/: 62 archivos, uno por consulta, agrupados por ejercicio, más las vistas (sql/views) y la materialización (sql/build)
Notebooks	notebooks/03, 04, 05, 06 y 08, guardados con sus resultados
Documentación de las consultas	docs/01 a docs/09: objetivo, fuente, resultado y decisión de cada consulta
Benchmark	scripts/build_database.py, scripts/benchmark.py y resultados en docs/06_benchmark/
Indicadores y visualizaciones	sql/07_indicators/, scripts/metabase_dashboard.py y scripts/charts.py
Evidencia del tablero	docs/07_dashboard/: capturas con dos y tres años y dashboard.json
Instrucciones de reproducción	README.md

2. Resultados por ejercicio

Ejercicio 1, ambiente. Docker Compose levanta JupyterLab (Python 3.11, DuckDB 1.5.5) y Metabase v0.63 con el driver de DuckDB. Se verificaron ambos servicios y el procedimiento quedó en el README.

Ejercicio 2, descarga. El script se parametrizó por años, reintenta las consultas al servidor, compara el tamaño descargado con el publicado y tiene un modo de verificación. Se obtuvieron los 16 archivos de 2026 (enero a agosto). La completitud se comprobó con tres criterios: meses publicados por la TLC, tamaño exacto y metadatos Parquet legibles.

Ejercicio 3, Parquet directo. Se consultaron 30.0 millones de viajes de 2026 sin importarlos, con `read_parquet` y funciones de metadatos. Los esquemas difieren entre tipos de taxi y entre meses. Los problemas principales: el 26 % de los amarillos no trae pasajeros ni tarifa, hay fechas desde 2001, distancias imposibles, montos negativos y un proveedor sin hora de descenso. La vista `trips_clean` descarta el 3.7 % de los amarillos y el 4.3 % de los verdes.

Ejercicio 4, análisis exploratorio. Se plantearon 13 preguntas sobre tiempo, características del viaje, tipos de taxi, pago, distribuciones y atípicos. Por ejemplo, los viajes de aeropuerto son el 8 % de los viajes amarillos pero generan el 21 % del ingreso, y el taxi amarillo es un servicio de Manhattan (87 % de los abordajes), mientras que el verde se concentra en East Harlem (40 %). Los cinco hallazgos están en `docs/04_ed.md`.

Ejercicio 5, 2024. Se incorporó cambiando una línea del script (40 archivos). Las 31 consultas anteriores corrieron sin errores; solo dos se ajustaron porque su resultado mezclaba años.

Ejercicio 6, benchmark. Se materializaron 72 millones de viajes en 7.5 s; la tabla ocupa 1.7 veces lo que el Parquet. Con un mes de datos la tabla fue entre 2.4 y 4.6 veces más rápida en la mayoría de las consultas; con 72 millones la ventaja bajó a 1.0–1.8 veces. Buscar un solo día fue entre 8 y 11 veces más rápido con la tabla (figura 1).

Ejercicio 7, tablero. En Metabase se construyó un tablero de 11 indicadores y 3 KPI que responde 12 preguntas. Se crea por código con la API de Metabase (figura 2).

Ejercicio 8, 2025. Con 2025 se llegó a 64 archivos y 121.2 millones de viajes. Las 37 consultas y las 12 del tablero siguieron funcionando sin cambios; el tablero se actualizó reconstruyendo la base (figuras 3 y 4).

Ejercicio 9, discusión. Las respuestas de 9.1 a 9.8 están en `docs/09_discussion.md`; lo esencial se resume en las secciones 3 y 4.

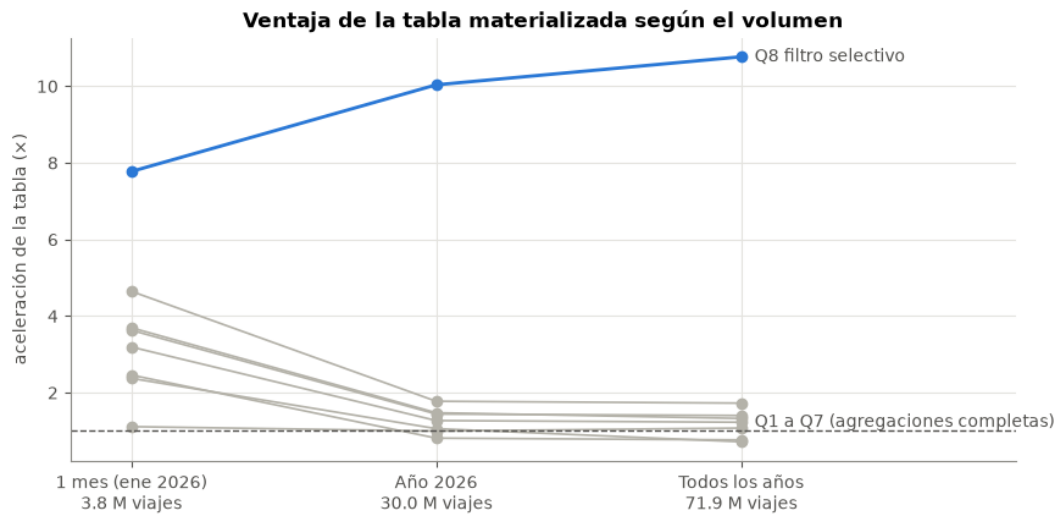


Figura 1. Aceleración de la tabla materializada frente a Parquet según el volumen de datos.

Lab 8 - Viajes de taxi en Nueva York

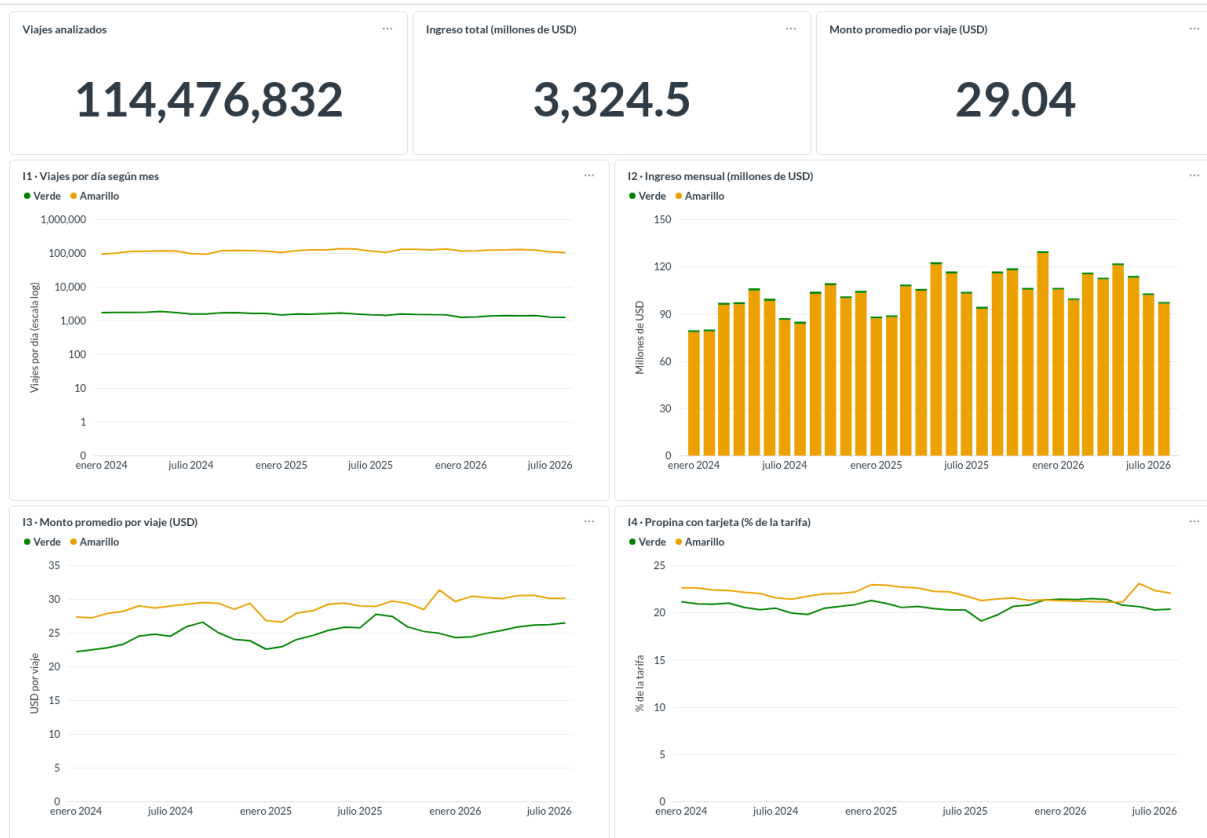


Figura 2. Tablero en Metabase con 2024, 2025 y 2026: KPI, viajes por día, ingreso, monto promedio y propina.

3. Hallazgos importantes de los datos

- 2025 fue el año de crecimiento. Entre enero y agosto, los viajes amarillos crecieron 13.6 % frente a 2024 (19.6 % con los datos crudos). En 2026 el volumen se estabilizó y quedó por debajo de 2025 en seis de los ocho meses.
- Los taxis verdes pierden terreno de forma sostenida: bajaron en los 20 meses comparables y su participación pasó de 1.61 % a 1.28 % y a 1.12 %.
- Cambió el registro del pago. Los viajes flex fare, que no informan medio de pago ni propina, pasaron de 4 % a cerca de 28 % de los viajes amarillos; la tarjeta bajó de 80 % a 63 %.
- El cargo de congestión de Manhattan (desde enero de 2025) se cobra en cerca del 72 % de los viajes amarillos y recauda unos 2 millones de USD al mes. El monto promedio por viaje subió de 28.25 a 30.14 USD.
- Los aeropuertos generan mucho más ingreso que viajes, pero su peso baja: 10.0 % de los viajes y 27.6 % del ingreso en 2024, contra 8.1 % y 20.9 % en 2026.
- La calidad depende del proveedor y del año. El proveedor 7 no registra la hora de descenso: la limpieza inicial eliminaba sus 367 mil viajes y se corrigió en un commit aparte. En 2025 aparecieron 1.88 millones de registros flex fare con tarifa negativa.

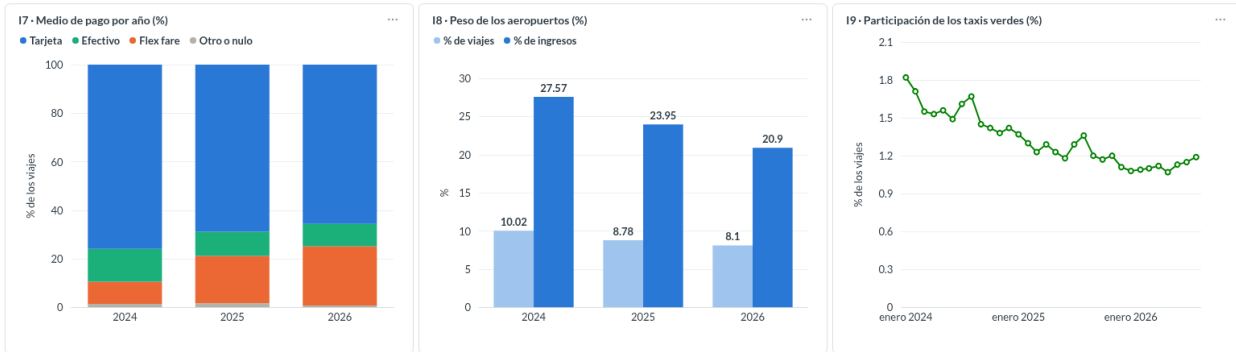


Figura 3. Medio de pago por año, peso de los aeropuertos y participación de los taxis verdes.

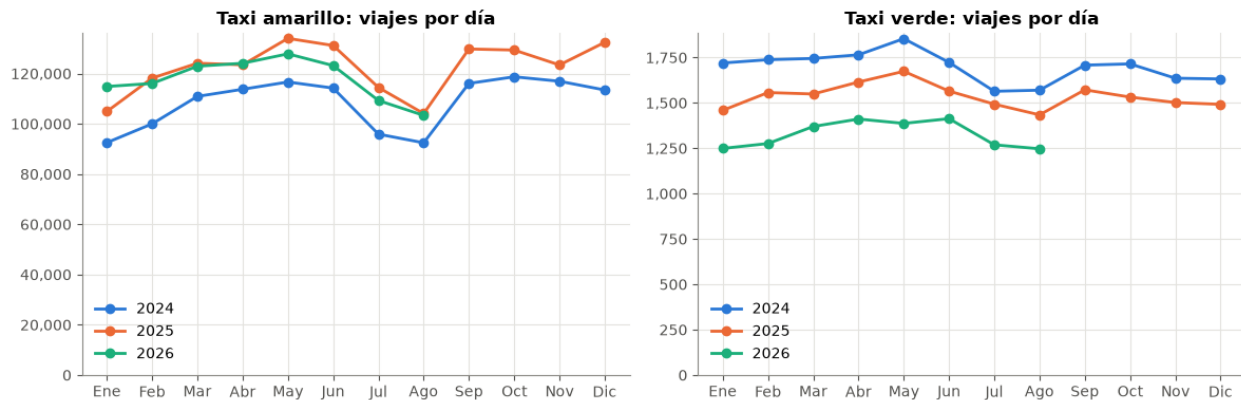


Figura 4. Viajes por día, mes a mes, en 2024, 2025 y 2026.

4. Aprendizajes

- Consultar Parquet directamente con globs y union_by_name permitió agregar dos años completos sin modificar el SQL: la estructura de carpetas y las vistas absorben los archivos y las columnas nuevas.
- Materializar no siempre conviene. Paga en consultas cortas y repetidas y en filtros selectivos, como los del tablero, pero casi no mejora las agregaciones pesadas, duplica el espacio y hay que reconstruir la tabla con cada carga.
- Con datos grandes la calidad cambia con el tiempo y con el proveedor. Una regla de limpieza razonable puede borrar segmentos completos sin que se note, por eso hay que revisar los descartes por año y por proveedor.
- La memoria es una restricción de diseño. Una consulta de duplicados agotó 4 GB y se resolvió agrupando por un hash de la fila; los percentiles exactos tardaron 37 s y los aproximados menos de un segundo.
- Las conclusiones dependen del periodo. Con 2024 y 2026 parecía un crecimiento continuo; 2025 mostró que fue un salto seguido de estabilidad.
- La reproducibilidad se diseña desde el inicio: ambiente en Docker con versiones fijas, datos fuera de Git, SQL en archivos, scripts idempotentes y tablero generado por código.

5. Conclusiones

DuckDB permitió analizar 121 millones de viajes en una laptop, sin servidor de base de datos y con consultas de segundos. El flujo construido descarga, verifica, consulta, materializa y visualiza de forma reproducible, y crece con cambios mínimos cuando llegan datos nuevos. La combinación más útil fue usar Parquet directo para explorar e incorporar datos, y una tabla materializada para el tablero. Más allá del rendimiento, el aprendizaje principal fue validar la calidad de forma continua: los problemas más importantes solo aparecieron al mirar varios años y varios proveedores a la vez.